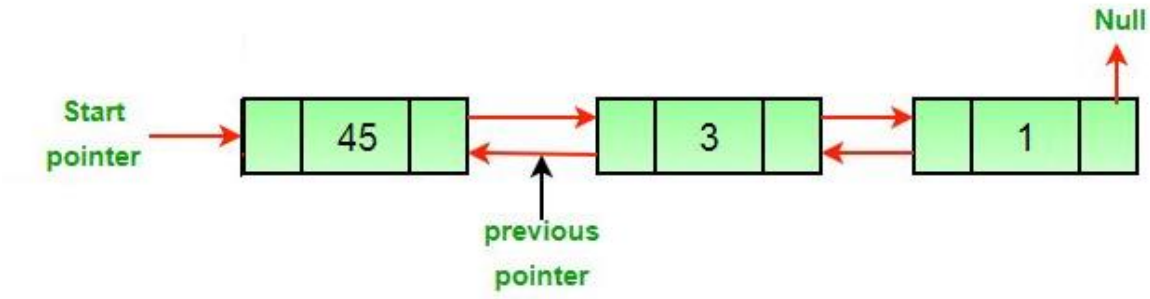

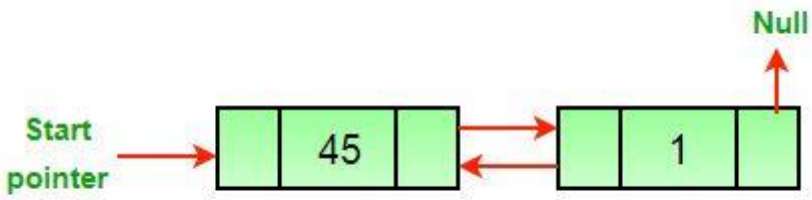

1. Delete Node

```
struct Node {
    int data;
    struct Node* next;
    struct Node* prev;
};
void push(struct Node** head_ref, int new_data)
{
    /* allocate node */
    struct Node* new_node = (struct Node*)malloc(sizeof(struct Node));

    /* put in the data  */
    new_node->data = new_data;

    /* since we are adding at the beginning,
    prev is always NULL */
    new_node->prev = NULL;

    /* link the old list off the new node */
    new_node->next = (*head_ref);

    /* change prev of head node to new node */
    if ((*head_ref) != NULL)
        (*head_ref)->prev = new_node;

    /* move the head to point to the new node */
    (*head_ref) = new_node;
}

void deleteNode(struct Node** head_ref, struct Node* del)
{
    /* base case */
    if (*head_ref == NULL || del == NULL)
        return;

    /* If node to be deleted is head node */
    if (*head_ref == del)
        *head_ref = del->next;
```

```
    /* Change next only if node to be deleted is NOT the last node */
    if (del->next != NULL)
        del->next->prev = del->prev;

    /* Change prev only if node to be deleted is NOT the first node */
    if (del->prev != NULL)
        del->prev->next = del->next;

    /* Finally, free the memory occupied by del*/
    free(del);
    return;
}
```

Complexity Analysis:

- **Time Complexity:** O(1).
  Since traversal of linked list is not required so the time complexity is constant.
- **Space Complexity:** O(1).
  As no extra space is required, so the space complexity is constant.

2. Delete a Doubly Linked List node at a given position

```
   void deleteNodeAtGivenPos(struct Node** head_ref, int n)
   {
       /* if list in NULL or invalid position is given */
       if (*head_ref == NULL || n <= 0)
           return;

       struct Node* current = *head_ref;
       int i;

       /* traverse up to the node at position 'n' from
          the beginning */
       for (int i = 1; current != NULL && i < n; i++)
           current = current->next;

       /* if 'n' is greater than the number of nodes
          in the doubly linked list */
       if (current == NULL)
           return;

       /* delete the node pointed to by 'current' */
       deleteNode(head_ref, current);
   }
void deleteNode(struct Node** head_ref, struct Node* del)
{
    /* base case */
    if (*head_ref == NULL || del == NULL)
        return;

    /* If node to be deleted is head node */
    if (*head_ref == del)
        *head_ref = del->next;

    /* Change next only if node to be deleted is NOT
       the last node */
    if (del->next != NULL)
        del->next->prev = del->prev;
```

```
    /* Change prev only if node to be deleted is NOT
       the first node */
    if (del->prev != NULL)
        del->prev->next = del->next;

    /* Finally, free the memory occupied by del*/
    free(del);
}
```

3. Swap Kth node from beginning with Kth node from end in a Doubly Linked List

Input: *DLL = 1 <-> 2 <-> 3 <-> 4 <-> 5, K = 1*
Output: *5 2 3 4 1*

Let **K1** be the **K**th node from beginning and **K2** be **K**th node from ending. Then:

- The previous node to **K2** has to be changed to the previous node of **K1**.
- The next node to **K2** has to be changed to the next node of **K1**.
- The previous node to **K1** has to be changed to the previous node of **K2**.
- The next node to **K1** has to be changed to the next node of **K2**.

4. Delete all occurrences of a given key in a doubly linked list

```
delAllOccurOfGivenKey(head_ref, x)
      if head_ref == NULL
          return
      Initialize current = head_ref
      Declare next
      while current != NULL
           if current->data == x
               next = current->next
               deleteNode(head_ref, current)
               current = next
           else
               current = current->next
```

5. Delete all the nodes from the doubly linked list that are greater than a given value

6. Delete all the even nodes from a Doubly Linked List

```
void deleteEvenNodes(Node** head_ref)
{
    Node* ptr = *head_ref;
    Node* next;
```

```
    while (ptr != NULL) {
        next = ptr->next;
        // if true, delete node 'ptr'
        if (ptr->data % 2 == 0)
            deleteNode(head_ref, ptr);
        ptr = next;
    }
}
void deleteNode(Node** head_ref, Node* del)
{
    // base case
    if (*head_ref == NULL || del == NULL)
        return;

    // If node to be deleted is head node
    if (*head_ref == del)
        *head_ref = del->next;

    // Change next only if node to be
    // deleted is NOT the last node
    if (del->next != NULL)
        del->next->prev = del->prev;

    // Change prev only if node to be
    // deleted is NOT the first node
    if (del->prev != NULL)
        del->prev->next = del->next;

    // Finally, free the memory occupied by del
    free(del);

    return;
}
```

7.

8.

9.

10.

11.

12.

13.

14.

15.

Reverse a Doubly Linked List

```
void reverse(struct Node **head_ref)
{
     struct Node *temp = NULL;
     struct Node *current = *head_ref;

     /* swap next and prev for all nodes of
       doubly linked list */
     while (current !=  NULL)
     {
       temp = current->prev;
       current->prev = current->next;
       current->next = temp;
       current = current->prev;
     }

     /* Before changing head, check for the cases like empty
        list and list with only one node */
     if(temp != NULL )
        *head_ref = temp->prev;
}
```

Current->prev	Current	Current->next	temp	head
null	1	2		
2		Null	Null	
3	2	1	1	
Null	3	2	2	
	Null			

1 2 3